Astana IT University
School of Software Engineering

Diploma Project

Development of a System for Detecting and Anonymizing Personal Data Before Sending Text to Artificial Intelligence Services

System Design Diagrams

Authors: Dossan Amina, Abzhamal Nazerke
Educational programme: Software Engineering
Supervisor: Sakhipov A.A., PhD,
Assistant Professor

Astana, 2026

Contents

1 Introduction **3**

2 System Context **3**

3 Component Diagram (Draft) **3**

4 Sequence of the Main Scenario **5**

5 Use Cases **6**

6 Database **6**

1. Introduction

This document presents the design diagrams of the PII Anonymization Gateway: the system context diagram, the component diagram (draft), the sequence diagram of the main scenario, the use case diagram and the entity-relationship (ER) diagram of the database. The diagrams are based on the documents *Requirements Specification* and *Technology Comparison* and are consistent with the user interface prototype. The editable sources (`.drawio`) and the exported images (`.png`) are stored in the `diagrams` directory of the project repository.

2. System Context

Figure 1 shows the system as a single element and the people and external systems that interact with it. Users send text with personal data and receive restored responses; information security specialists configure policies and review the log; internal systems use the API proxy; the organization's IT department deploys the server and installs the browser extension by browser policy. ChatGPT and Gemini are external systems that receive only anonymized text.

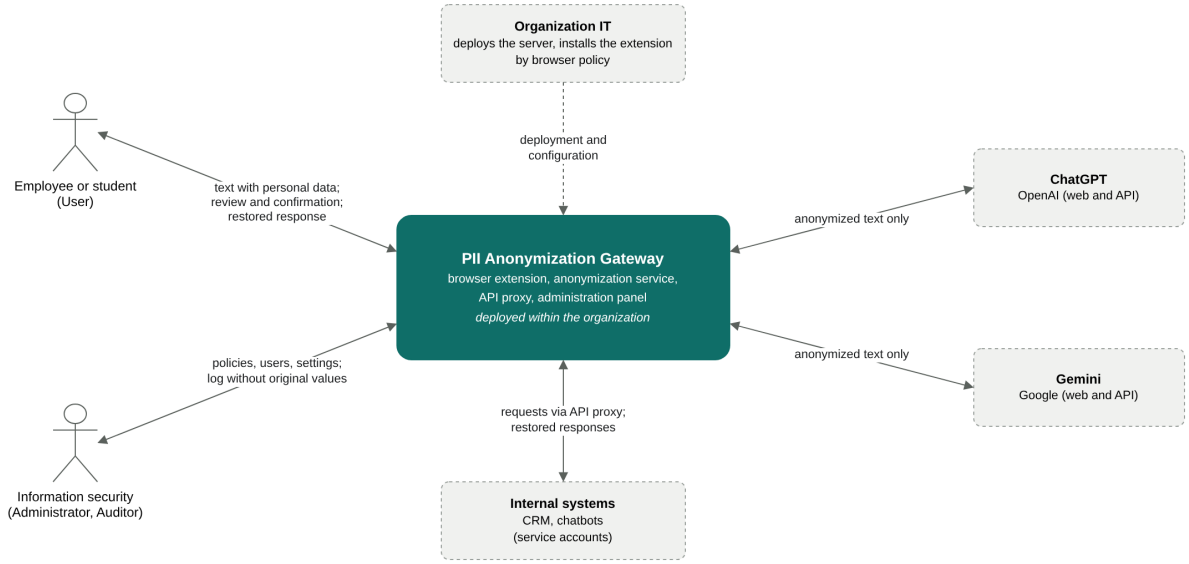


Figure 1: System context diagram

3. Component Diagram (Draft)

Figure 2 shows the components of the system and their deployment. All components that process original personal data are deployed within the organization's infrastructure using Docker Compose (NFR-3); only anonymized text crosses the boundary of the organization (NFR-2).

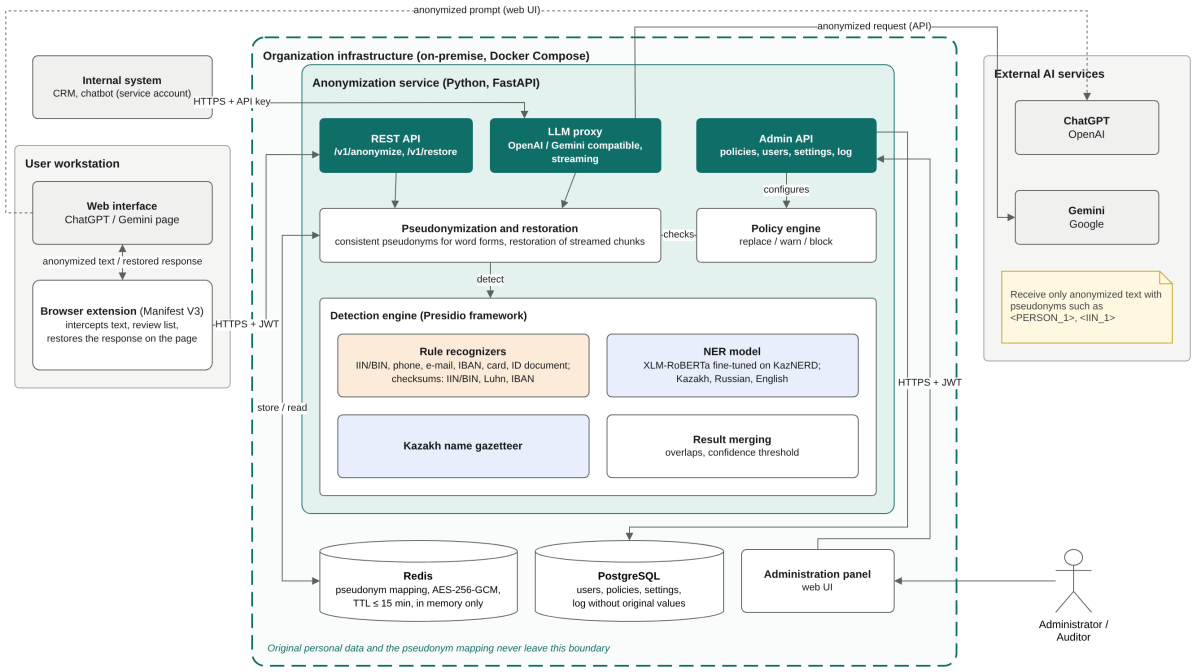


Figure 2: Component diagram of the system (draft)

The system consists of the following components:

- *Browser extension* (Manifest V3) intercepts the text in the web interfaces of ChatGPT and Gemini, sends it to the anonymization service, shows the detected data for review and restores the response on the page (FR-5).
- *Anonymization service* (Python, FastAPI) provides three interfaces: the REST API for the extension, the LLM proxy compatible with the OpenAI and Gemini interfaces for service accounts (FR-8), and the Admin API for the administration panel.
- *Detection engine* is built on the Presidio framework and combines rule recognizers with checksum validation, the XLM-RoBERTa NER model fine-tuned on KazNERD and the Kazakh name gazetteer; the results are merged using the confidence threshold (FR-1, FR-2).
- *Pseudonymization and restoration module* assigns consistent pseudonyms and restores original values, including streamed responses (FR-3, FR-4); the *policy engine* applies the actions replace, warn and block (FR-6).
- *Redis* stores the pseudonym mapping in memory, encrypted with AES-256-GCM, with a retention of at most 15 minutes (NFR-2).
- *PostgreSQL* stores users, roles, policies, settings and the log without original values (FR-7).
- *Administration panel* is a web interface for administrators and auditors (FR-7).

4. Sequence of the Main Scenario

Figure 3 shows the interaction of the components when a user sends a request containing personal data through the browser extension and receives a restored response (use cases UC-1 and UC-2).

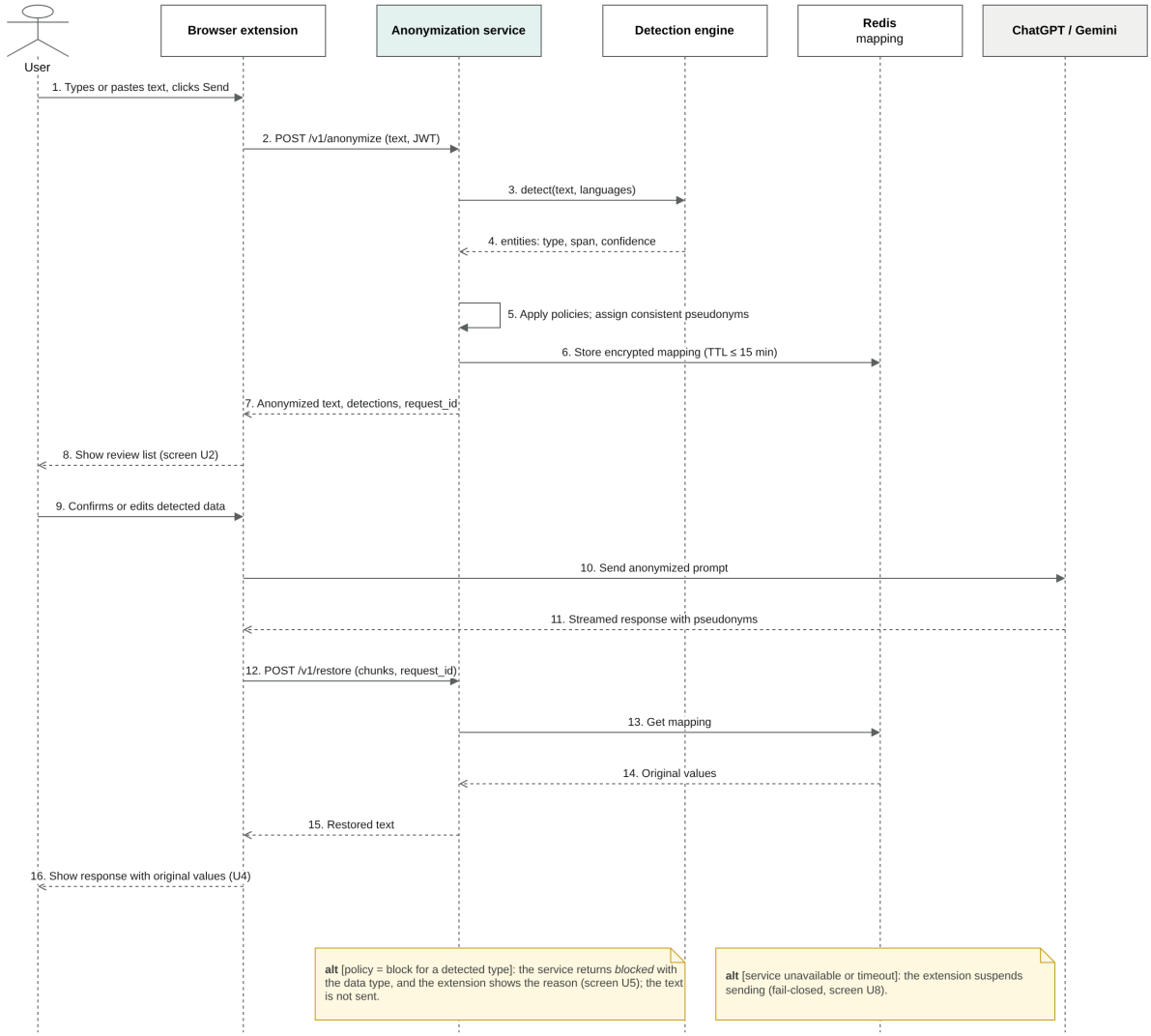


Figure 3: Sequence diagram of sending a request and restoring the response

The original text is sent only to the anonymization service within the organization (step 2). The AI service receives only the anonymized text (step 10), and the original values are restored using the mapping stored in Redis (steps 12–15). If a detected data type has the policy "block", the text is not sent and the user sees the reason; if the anonymization service is unavailable, sending is suspended (NFR-4).

5. Use Cases

Figure 4 shows the actors of the system and the main use cases defined in the requirements specification.

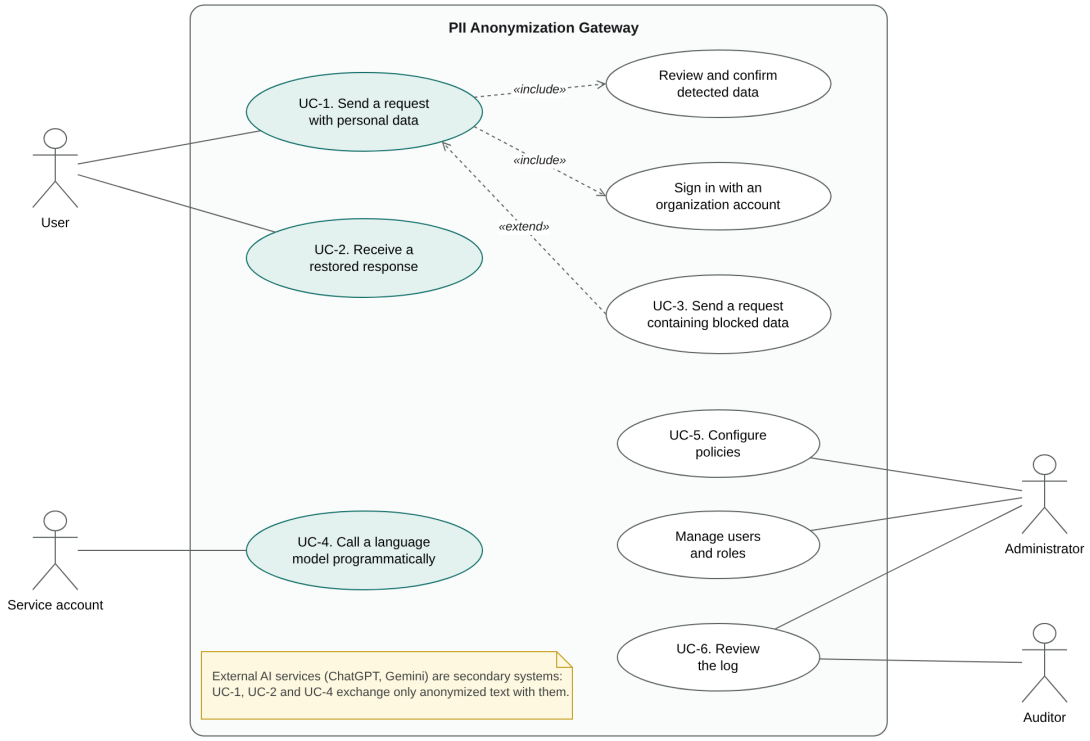


Figure 4: Use case diagram

The user sends requests and receives restored responses (UC-1, UC-2); reviewing the detected data and signing in with an organization account are included in UC-1, and sending a request with blocked data (UC-3) extends it. The service account calls a language model through the API proxy (UC-4). The administrator configures policies, manages users and roles and reviews the log (UC-5, UC-6); the auditor has read-only access to the log.

6. Database

Figure 5 shows the structure of the PostgreSQL database.

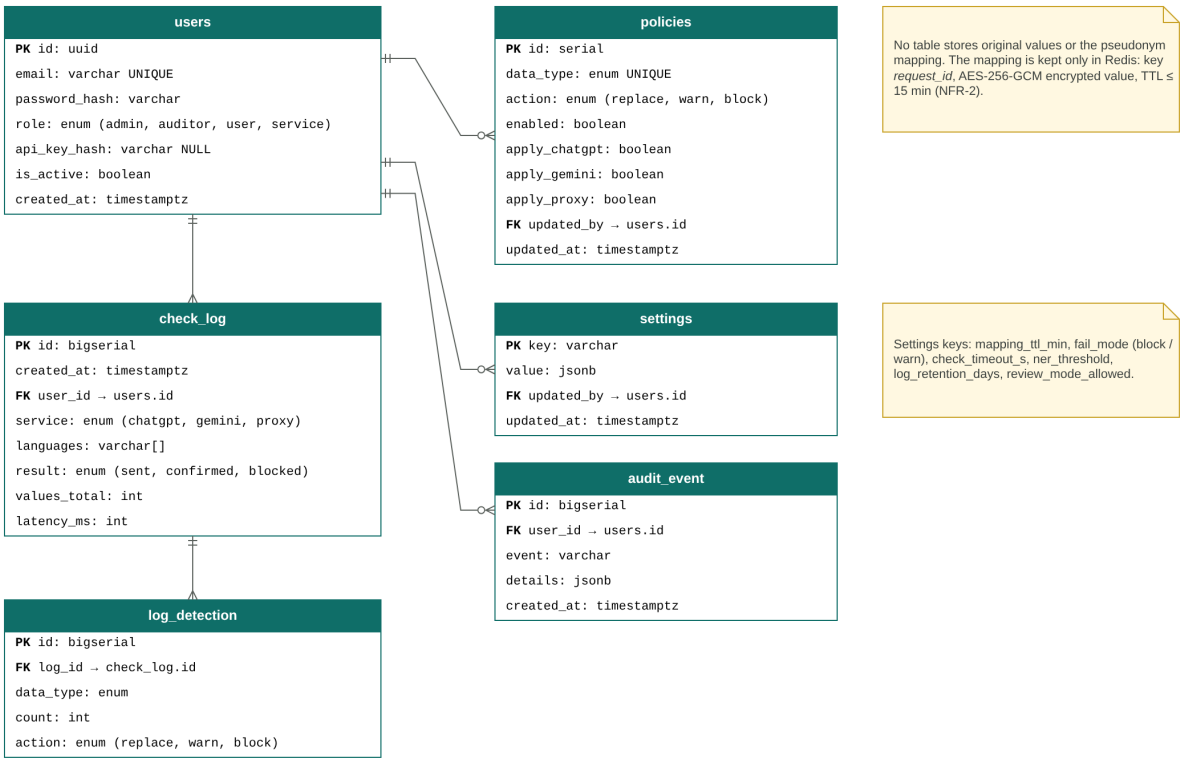


Figure 5: Entity-relationship diagram of the database

The table `users` stores accounts of all roles, including service accounts with a hashed API key. The table `policies` contains one record per data type with the selected action and the services to which it applies (FR-6). The table `settings` stores system parameters such as the mapping retention period, the behaviour on failure and the NER confidence threshold (FR-7.5). Each check is recorded in `check_log`, and the number of detected values of each type in `log_detection` (FR-7.1). Changes made by administrators are recorded in `audit_event`. No table contains original values or the pseudonym mapping: the mapping is stored only in Redis and is deleted automatically (NFR-2).